

Every program has two parts:
serial portion - must run sequentially (can't be parallelized)
parallel portion - can be split across multiple processors simultaneously

S = prop of program that is serial
N = number of processors

speedup = 1 / (**S** + (1-**S**)/**N**) x

ex: program is 30% serial, 70% parallel. Run on 4 processors:
speedup = 1 / (0.3 + 0.7/4)
= 1 / (0.3 + 0.175) = 1 / 0.475
= **2.11x**

max speedup (inf. processors) = 1/**S**

Amdahl's Law
less ser portion → more possible speedup
(reducing **S** more valuable than increasing **N**)

HDD (Hard Disk Drive) - stores data on spinning magnetic platters
- read/write head moves across the platter to read or write data

seek time - time to move the head to the right track
rotational latency - time waiting for the right sector to spin under the head
transfer time - time to actually read/write the data

total access time = seek time + rotational latency + transfer time

avg rotational latency = (1/2) × (1/RPM) × 60 seconds

ex: Disk spins at 7200 RPM:
time per rotation = 1/7200 minutes = 60/7200 seconds = 8.33 ms
average rotational latency = 8.33/2 = 4.17 ms

	HDD	SSD
Storage	magnetic platters	flash memory chips
moving parts	yes (head platters)	no
seek time	~5-10ms	~0.1ms
durability	vulnerable to shock	more durable
cost per GB	cheaper	more expensive

SSDs have no seek time or rotational latency — access is purely electronic. This is why SSDs feel so much faster for random access.

Fragmentation - when files are stored in non-contiguous chunks scattered across the disk.
Internal fragmentation - allocated space is larger than needed (wasted space inside a block)
External fragmentation - enough total free space exists but it's scattered in small chunks that can't fit a large file
ex: [FILE A][free][FILE B][free][FILE C][free]
2KB 2KB 2KB → 6KB free total
But you need a 5KB contiguous block → can't fit!

- HDDs suffer more from fragmentation than SSDs because physical location matters for HDDs (head has to move). SSDs access any location equally fast.

Row storage - store all fields of one record together:
[name, age, salary] [name, age, salary] [name, age, salary]

Columnar storage - store all values of one field together:
[name, name, name] [age, age, age] [salary, salary, salary]

	Row Storage	Columnar Storage
Best for	Accessing full records (e.g. fetch one user's data)	Analyzing one field across many records (e.g. average salary)
Example	Traditional databases (MySQL)	Analytics (Google BigQuery)
Why faster for analytics	Only load the columns you need	Must load entire rows even if you only need one field

Row storage = good for **OLTP** (transactional, fetch whole records)
Columnar storage = good for **OLAP** (analytics, aggregate one field across millions of rows)

each CPU instruction goes through multiple stages:

IF — Instruction Fetch (get the instruction from memory)
ID — Instruction Decode (figure out what it means)
EX — Execute (do the operation)
MEM — Memory access (read/write if needed)
WB — Write Back (store the result)

w/o **pipelining** → each instruction completes all 5 stages before the next one starts
w/ **pipelining** → we overlap stages like an assembly line
Time: 1 2 3 4 5 6 7 8 9
Inst 1: [IF] [ID] [EX][MEM] [WB]
Inst 2: [IF] [ID] [EX][MEM] [WB]
Inst 3: [IF] [ID] [EX][MEM] [WB]
Inst 4: [IF] [ID] [EX][MEM] [WB]

Ideal speedup = number of pipeline stages

Time to complete N instructions (pipelined) = (# of stages + N - 1) × cycle time

ex:
- 5-stage pipeline, 1000 instructions, Each stage takes 1 ns
Without pipelining: 1000 × 5 = 5000 ns
With pipelining: (5 + 1000 - 1) × 1 = 1004 ns
Speedup ≈ 5000/1004 ≈ 4.98x ≈ 5x

speedup approaches the # of stages as N inc - the startup cost (stages - 1) becomes negligible.

sometimes instructions depend on each other, causing hazards — situations that stall the pipeline.

Haza rd	Cause	Example
Struct ural	Two instructions need the same hardware at the same time	Two instructions both need the memory unit
Data	One instruction needs the result of a previous one that hasn't finished	ADD R1, R2 then MUL R3, R1 — R1 not ready yet
Contr ol	A branch instruction changes the flow — fetched instructions may be wrong	if (x > 0) goto Y — don't know next instruction until EX

Speedup w/ stalls = N stages / (1 + stall cycles per instruction)
ex: 5-stage pipeline, 20% of instructions cause a 2-cycle stall, No forwarding
Stall cycles per instruction = 0.2 × 2 = 0.4

Speedup = 5 / (1 + 0.4) = 5/1.4 = 3.57x

memory hierarchy
cpu → register → L1 cache → L2 cache → ram → hard drive

unit	size	ex
bit	1 bit	0 or 1
byte	8 bits	one character
kilobyte (kb)	2 ¹⁰ = 1024 bytes	short text file
megabyte (mb)	2 ²⁰ ≈ 1mil bytes	photo
gigabyte (gb)	2 ³⁰ ≈ 10 ⁹ bytes	movie
terabyte (tb)	2 ⁴⁰ ≈ 10 ¹² bytes	large hard drive
petabyte (pb)	2 ⁵⁰ ≈ 10 ¹⁵ bytes	netflix's library

2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
-128	64	32	16	8	4	2	1
1	0	1	1	1	0	0	1
-128	0	32	16	8	0	0	1

= -128+0+32+16+8+0+0+1 = -71

□ 8-bit **range**: -128 → 127 □ 16-bit **range**: -32,768 → 32,767

negating a number → flip 1s & 0s then add 1
ex: 5 (0000 0101) → -5 (1111 1010 + 1 = **1111 1011**)

overflow ex: **155** = 128+16+8+2+1 = 1001 1011 → -128+0+0+0+16+8+0+2+1 = **-101**

hex → binary
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A (10), B (11), C (12), D (13), E (14), F (15)
0011 → 0x3 **1010** → 0xA **1100** → 0xC **1111** → 0xF

16 ³	16 ²	16 ¹	16 ⁰
4096	256	16	1
3	A	C	F
12288	2560	192	15

= 12288 + 2560 + 192 + 15 = 15055

decimal → hex
divide repeatedly by 16, collect remainders bottom to top:
185 ÷ 16 = 11 remainder **9**
11 ÷ 16 = 0 remainder 11 (= **B**)
read remainders bottom to top: B, 9 → 0xB9

	bits	# sign bits	# exp bits	# frac bits	bias
FP32	32	1	8	23	127
FP16	16	1	5	10	15
BF16	16	1	8	7	127

bias = 2ⁿ⁻¹ - 1 (n = # exp bits)

FP ex: 0 **10 101**
sign bit → 0 → pos (-1⁰ = 1)
exp bits → **10**: = 2 (stored exp value)
act exp value = 2 - **1** = **1** → 2¹ = **2** (bc binary uses base 2)
frac bits → 1.101₂ = 1 + 2⁻¹ + 0 + 2⁻³ = 1 + 0.5 + 0.125 = **1.625**
all together: **1 * 2 * 1.625 = 3.25**

measuring cpu speed → Hz (clock cycles / second)
1 GHz = 1bil cycles per second = 10⁹ cycles/sec
□ modern CPU clock speed ≈ 2-4 GHz

avg CPI (cycles per instruction) ex: program =
- 50% arithmetic instructions (1 cycle each)
- 20% control instructions (3 cycles each)
- **30% memory instructions** (2 cycles each)
avg CPI = 0.5(1) + 0.2(3) + **0.3(2)** = **1.7 CPI**

CPI w/ stalls = **CPI w/o stalls** + (prop mem inst * # stall cycles per mem inst) CPI
1 mem inst stalls for 40 cycles → CPI = **1.7+0.3(40)** = **13.7 CPI**

cache reduces stall from 40 cycles → 10 cycles:
new CPI = **1.7+0.3(10)** = **4.7 CPI**

speedup = Old CPI ÷ New CPI x
= 13.7 ÷ 4.7 = **2.91x** (cache made prog ~3x faster)

total cycles = # instructions * avg CPI cycles
execution time = **total cycles** + clock speed sec
stall cycles = (# instructions * **prop mem instructions** * # stall cycles / mem instruction) cycles
= 1B * **0.3** * 40 = **12B cycles**
prop stall cycles = # stall cycles ÷ **total cycles** %

memory access → cache hit (data in cache) or cache miss
hit rate = # cache hits / # memory accesses

AMAT (avg mem access time) = (**hit rate** * cache time) + (**miss rate** * RAM time) sec

3-level AMAT = L1 time + **L1 miss rate**(L2 time + **L2 miss rate**(RAM time)) sec

prop reaching RAM = **L1 miss rate** * **L2 miss rate** %
speedup = old AMAT / new AMAT x

□ typical cache line size = 64 bytes
elems per cache line = cache line size + element size
- **float64** (8 bytes) → 64 ÷ 8 = **8 floats** / cache line
- **float32** (4 bytes) → 64 ÷ 4 = **16 floats** / cache line
- **int32** (4 bytes) → 64 ÷ 4 = **16 ints** / cache line

1	2	3
4	5	6
7	8	9

row major: **1 2 3 4 5 6 7 8 9** col major: **1 4 7 2 5 8 3 6 9**

rm: access A[0][0] → A[0][0] ... A[0][2] loaded into cache → next 2 accesses are cache hits → **miss rate** ≈ 1/2 (1 miss per cache line load)

cm: access A[0][0] → A[0][0] ... A[2][0] loaded into cache (further in memory, separated by a row) → every access is cache miss → **miss rate** ≈ 100%

byte-addressable - each byte has an address (most comm)
word-addressable - each word (ex 4 bytes) has an address

addressable locations = mem size + addressability unit = 2ⁿ (n = width of mem address in bits)

ex: 16 GB of word-addressable memory → 16*10⁹ bytes ÷ 4 bytes = **4*10⁹ addressable locations**

address space size = 2ⁿ * addressability unit
ex: 32-bit byte-addressable system → 2³² * 1 byte ≈ 4 × 10⁹ bytes = **4 GB**

alignment - a piece of data must start at an address that's a multiple of its size (in bytes)

ex: a float32 (4 bytes) must start at an address that's a mult of 4

data type	size
byte / char	1 byte
short	2 bytes
int / float32	4 bytes
double / float64	8 bytes

□ instructions usually 4 bytes wide

padding - empty bytes to help with alignment
reordering variables can minimize padding & save space

OS - software that hands out hardware resources (CPU, memory, disk, network) to programs that need them, also prevents programs from crashing each other

process - program loaded into memory & actively executing

- **code** (instructions)
- **data** (variables, heap)
- **stack** (function calls, local variables)

- **program counter** (next instructions)
- **register values** (current state of computation)

□ one program can spawn many processes

process states

- **new** - process just created, not ready to run

- **ready** - has everything it needs, waiting for CPU

- **running** - currently executing on CPU

- **waiting** - CPU avail but waiting for sth (I/O, disk read, ntwk)

- **terminated** - finished executing

- **new** → ready: admit

- **OS dispatches** the process (gives it to CPU)
- running → waiting: process gives up CPU to wait for something

- waiting → ready: ready to run again, waiting its turn

- **PCB** (Process Control Block) - OS's data structure that stores everything about a process (state, registers, memory addresses, etc.)

- **context switch** - OS switching from one process to another

1. save current process' state in PCB
2. load next process' saved state
3. hand CPU to next process

□ takes time where no useful work is being done

- **fork()** - system call that creates a copy of the current process

- a new **child** process is created (almost exact copy of parent)

- both processes continue from same point (line after fork())

- only diff is return value of fork()

processes doubles after fork()

□ helps to draw a tree

- n unconditional forks → 2ⁿ processes

exec() - replaces current process with a brand new program

process	arrival	burst
P1	0	7
P2	2	4
P3	4	1
P4	5	4

P1: comp=16 turnaround=16-0=16 waiting=16-7=9
P2: comp=7 turnaround=7-2=5 waiting=5-4=1
P3: comp=5 turnaround=5-4=1 waiting=1-1=0
P4: comp=11 turnaround=11-5=6 waiting=6-4=2

- **RR** (Round Robin) - each process uses CPU for time quantum, then goes to back of queue

- process arrives at same time quantum expires → new quantum, then goes to back of queue

- best for **interactive systems, fairness** (no starvation)

- preemptive
- smaller quantum → more time spent switching

- longer quantum → approaches FCFS

process	arrival	burst
P1	0	5
P2	1	3
P3	2	8

time quantum = 2
P1: comp=12 turnaround=12-0=12 waiting=12-5=7
P2: comp=9 turnaround=9-1=8 waiting=8-3=5
P3: comp=16 turnaround=16-2=14 waiting=14-8=6

virtual addresses - addresses in a process' own private imaginary memory space
- the actual location in RAM
- **MMU** (mem mgmt unit) - translates virtual address → physical address
- programs can't overwrite other programs or access their data
- OS stores **overflow** (inactive pages) on disk

turnaround time = completion time - arrival time

waiting time = turnaround time - burst time

- **FCFS** (first come first serve)

- best for **simple batch jobs**

- not preemptive
- con: don't actually know burst times in advance

- **SJF** (shortest job first)

- best for **minimizing avg wait**

- preemptive
- con: don't actually know burst times in advance

- con: also prone to **starving**

virtual page #	physical frame #	valid?
0	3	yes
1	7	yes
2	—	no
3	1	yes

valid - page in RAM, translation works
not valid - page on disk, causes page fault

a **virtual address** has 2 parts:

- **VPN** - which page are we in?

- **offset** - where within that page?

- page size = 2^k bytes → offset uses k bits

ex: 4 KB = 4 × 1024 = 2¹² bytes → offset = 12 bits

full translation ex

- 16-bit virtual address space

- page size = 256 bytes

obj: translate virtual address: 0x013F

VPN = 16 - 8 = 8 bits

256 bytes = 2⁸ → offset = 8 bits

VPN = 0000 0001 = 1 → frame 5

Offset = 0011 1111 = 0x3F

frame 5 w/ offset 0x3F = frame 5 * page size + offset

= 5 * 256 + 63 = 1343

= 0x053F ← physical address

□ offset never changes → calculate physical frame # * page size

TLB - tiny, fast cache that stores recent page table entries

- cause now every mem access requires 2 mem accesses

(lookup page table in RAM & access actual data in RAM)

CPU generates virtual address → check TLB →

TLB miss → check page table in RAM → update TLB (slow)

TLB hit → use cached frame #

effective access time = TLB hit rate * (TLB time + memory time) + TLB miss rate * (TLB time + 2*memory time) s

pages = virtual address space size + page size

page table size = # pages × size of each entry

ex: - 32-bit virtual address space → 2³² bytes = 4 GB

- Page size = 4 KB = 2¹² bytes

- Each page table entry = 4 bytes

pages = 2³² + 2¹² = 2²⁰ = 1,048,576 pages

page table size = 2²⁰ * 4 bytes = 4 MB

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address

virtual mem divided into **pages**, physical mem divided into **frames**

of same size

maps page numbers to frame numbers

- CPU generates its own page table

- every process has its own page table

physical address